

- benefits of 262
 - building home theater system
 - about 257–259
 - constructing Facade in 263
 - implementing Facade class 260–262
 - implementing interface 264
 - class diagram 266
 - Complexity Hiding Proxy vs. 483
 - defined 266
 - exercises for 256, 275, 375, 379, 481, 574, 596
 - Principle of Least Knowledge and 271
 - factory method
 - about 125, 134
 - as abstract 135
 - declaring 125–127
 - Factory Method Pattern
 - about 131–132
 - about factory objects 114
 - Abstract Factory Pattern and 158–161
 - code up close 151
 - concrete classes and 134
 - creator classes 131–132
 - declaring factory method 125–127
 - defined 134
 - Dependency Inversion Principle 139–143
 - drawing parallel set of classes 133, 165
 - exercise matching description of 574, 596
 - interview with 158–159
 - looking at object dependencies 138
 - product classes 131–132
 - Simple Factory and 135
 - Factory Pattern
 - Abstract Factory
 - about 153
 - building ingredient factories 146–148, 167
 - combining patterns 502–505, 548
 - defined 156–157
 - exercise matching description of 574, 596
 - Factory Method Pattern and 158–160
 - implementing 158
 - exercise matching description of 300, 314
 - Factory Method
 - about 131–132
 - advantages of 135
 - code up close 151
 - creator classes 131–132
 - declaring factory method 125–127
 - defined 134
 - Dependency Inversion Principle 139–143
 - drawing parallel set of classes 133, 165
 - exercise matching description of 574, 596
 - looking at object dependencies 138
 - product classes 131–132
 - Simple Factory and 135
 - Simple Factory
 - about factory objects 114
 - building factory 115
 - defined 117
 - Factory Method Pattern and 135
 - pattern honorable mention 117
 - using new operator for instantiating concrete classes 110–113
 - Favor composition over inheritance design principle 23, 73, 75, 393
 - Fireside Chat
 - Decorator Pattern vs. Adapter Pattern 254–255
 - Strategy Pattern vs. State Pattern 414–415
 - Firewall Proxy 482
 - Flyweight Pattern 604–605
 - forces 568
 - for loop 344
 - frameworks vs. libraries 29
- ## G
- Gamma, Erich 587–588
 - Gang of Four (GoF) 569, 587–588
 - global access point 177
 - global variables, Singleton vs. 184
 - guide to better living with Design Patterns 564
 - gumball machine controller implementation, using State Pattern
 - cleaning up code 413
 - demonstration of 411–412
 - diagram to code 384–385
 - finishing 410
 - one in ten contest
 - about 390–391
 - annotating state diagram 396, 422
 - changing code 392–393

- drawing state diagram 391, 420
 - implementing state classes 397, 400–405, 409
 - new design 394–396
 - reworking state classes 398–399
- refilling gumball machine 416–417
- SoldState and WinnerState in 412
- testing code 388–389
- writing code 386–387
- gumball machine monitoring, using Proxy Patterns
 - about 426–428
 - Remote Proxy
 - about 429
 - adding to monitoring code 432
 - preparing for remote service 446–447
 - registering with RMI registry 448
 - reusing client for 449
 - reviewing process 453–454
 - role of 430–431
 - testing 450–452
 - wrapping objects and 468

H

- HAS-A relationships 23, 91
- HashMap 348, 352, 353
- hasNext() method 328, 342, 344
- Head First learning principles xxviii
- Helm, Richard 587–588
- high-level component classes 139
- The Hollywood Principle 298–300
- home automation remote control, using Command Pattern
 - about 193
 - building 203–205, 235
 - class diagram 207
 - creating commands to be loaded 208–209
 - defining 206
 - designing 195–196
 - implementing 210–212
 - macro commands
 - about 225
 - hard coding vs. 228
 - undo button 228, 236
 - using 226–227
 - mapping 201–202, 235

- Null Object 214
- testing 204, 212–213, 227
- undo commands
 - creating 217–219, 228
 - implementing for macro command 236
 - testing 220, 223–224
 - using state to implement 221
- vendor classes for 194
- writing documentation 215
- home theater system, building
 - about 257–259
 - constructing Facade in 263
 - implementing interface 264
 - Sharpen Your Pencil 270
 - using Facade Pattern 260–262
- hooks, in Template Method Pattern 293–295

I

- Image Proxy, writing 460–463
- implementations 13, 17, 43
- Implementation section, in pattern catalog 571
- implement on interface, in design patterns 117
- import and package statements 128
- inheritance
 - composition vs. 93
 - disadvantages of 5, 85
 - favoring composition over 23
 - for maintenance 4
 - for reuse 4, 13
 - implementing multiple 246
- instance variables 82–83, 97–98
- instantiating 110–113, 138, 170–172
- integrating Café Menu, using Iterator Pattern 347
- Intent section, in pattern catalog 571
- interface 11–12, 110–113
- interface type 15, 18
- internal iterators 342
- Interpreter Pattern 606–607
- Interview With
 - Composite Pattern 372–373
 - Decorator Pattern 104

- Factory Method Pattern and Abstract Factory Pattern 158–159
- Singleton Pattern 174
- inversion, in Dependency Inversion Principle 141
- invoker 201, 206–207, 209, 233
- IS-A relationships 23
- Iterable interface 343
- Iterator Pattern
 - about 327
 - class diagram 339
 - code up close using HashMap 348
 - code violating Open-Closed Principle 355–356
 - Collections and 353
 - combining patterns 507
 - defined 338–339
 - exercise matching description of 375, 379, 574, 596
 - integrating Café Menu 347
 - java.util.Iterator 334
 - merging diner menus
 - about 318–319
 - adding Iterators 328–334
 - cleaning up code using java.util.Iterator 335–337
 - encapsulating Iterator 325–326
 - implementing Iterators for 327
 - implementing of 320–325
 - removing objects 334
 - Single Responsibility Principle (SRP) 340–341
- Iterators
 - adding 328–334
 - allowing decoupling 333, 337, 339, 351–352
 - cleaning up code using java.util.Iterator 335–337
 - Collections and 353
 - encapsulating 325–326
 - Enumeration adapting to 251, 342
 - external 342
 - HashMap and 353
 - implementing 327
 - internal 342
 - ordering of 342
 - polymorphic code using 338, 342
 - using ListIterator 342
 - writing Adapter for Enumeration 252–253
 - writing Adapter that adapts to Enumeration 253, 275

J

- JavaBeans library 65
- Java Collections Framework 353
- Java decorators (java.io packages) 100–103
- Java Development Kit (JDK) 65
- Java Iterable interface 343
- java.lang.reflect package, proxy support in 440, 469, 476
- java.util.Collection 353
- java.util.Enumeration, as older implementation of Iterator 250, 342
- java.util.Iterator
 - cleaning up code using 335–337
 - interface of 334
 - using 342
- Java Virtual Machines (JVMs) 182, 432
- JButton class 65
- JFrames, Swing 308
- Johnson, Ralph 587–588

K

- Keep It Simple (KISS), in designing patterns 580
- Known Uses section, in pattern catalog 571

L

- lambda expressions 67
- Law of Demeter. *See* Principle of Least Knowledge
- lazy instantiation 177
- leaves, in Composite Pattern tree structure 360–362, 368
- libraries 29
- LinkedList 352
- ListIterator 342
- logging requests, using Command Pattern 230
- looping through array items 323
- Loose Coupling Principle 54

M

- macro commands
 - about 225
 - macro commands 228, 236
 - using 226–227
- maintenance, inheritance for, 4
- matchmaking service, using Proxy Pattern
 - about 470
 - creating dynamic proxy 474–478
 - implementing 471–472
 - protecting subjects 473
 - testing 479–480
- Mediator Pattern 543, 608–609
- Memento Pattern 610–611
- merging diner menus (Iterator Pattern)
 - about 318–319
 - adding Iterators 328–334
 - cleaning up code using java.util.Iterator 335–337
 - encapsulating Iterator 325–326
 - implementing Iterators for 327
 - implementing of 320–325
- method of objects, components of object vs. 267–271
- methods 143, 293–295
- modeling state 384–385
- Model-View-Controller (MVC)
 - about 520–521, 523–525
 - Adapter Pattern 540
 - Beat model 529, 549–552
 - Composite Pattern 526–527, 543
 - controllers per view 543
 - Heart controller 541, 561
 - Heart model 539
 - implementing controller 536–537, 556–557
 - implementing DJ View 528–535, 553–556
 - Mediator Pattern 543
 - model in 543
 - Observer Pattern 526–527, 531–533
 - song 520–521
 - state of model 543
 - Strategy Pattern 526–527, 536–537, 539, 558–560
 - testing 538
 - views accessing model state methods 543
- Motivation section, in pattern catalog 571

- multiple patterns, using
 - about 493–494
 - in duck simulator
 - about rebuilding 495–497
 - adding Abstract Factory Pattern 502–505, 548
 - adding Adapter Pattern 498–499
 - adding Composite Pattern 507–509
 - adding Decorator Pattern 500–501
 - adding Iterator Pattern 507
 - adding Observer Pattern 510–516
 - class diagram 518–519
- multithreading 181–182, 188

N

- Name section, in pattern catalog 571
- new operator
 - related to Singleton Pattern 171–172
 - replacing with concrete method 116
- next() method 328, 342, 344
- NoCommand, in remote control code 214
- nodes, in Composite Pattern tree structure 360–362, 368
- Null Objects 214

O

- object access, using Proxy Pattern for controlling
 - about 426–428
 - Caching Proxy 466, 482
 - Complexity Hiding Proxy 483
 - Copy-On-Write Proxy 483
 - Firewall Proxy 482
 - Protection Proxy
 - about 469
 - creating dynamic proxy 474–478
 - implementing matchmaking service 471–472
 - protecting subjects 473
 - testing matchmaking service 479–480
 - using dynamic proxy 469–470
 - Remote Proxy
 - about 429
 - adding to monitoring code 432
 - preparing for remote service 446–447
 - registering with RMI registry 448

- reusing client for 449
 - reviewing process 453–454
 - role of 430–431
 - testing 450–452
 - wrapping objects and 468
 - Smart Reference Proxy 482
 - Synchronization Proxy 483
 - Virtual Proxy
 - about 457
 - designing Virtual Proxy 459
 - reviewing process 465
 - testing 464
 - writing Image Proxy 460–463
 - object adapters vs. class adapters 246–249
 - object construction, encapsulating 600
 - object creation, encapsulating 114–115, 136
 - Object-Oriented (OO) design. *See also* design principles
 - adapters
 - about 238–239
 - creating Two Way Adapters 244
 - in action 240–241, 242
 - object and class object and class 246–249
 - design patterns vs. 30–31
 - extensibility and modification os code in 87
 - guidelines for avoiding violation of Dependency Inversion Principle 143
 - loosely coupled designs and 54
 - object patterns, Design Patterns 577
 - objects
 - components of 267–271
 - creating 134
 - loosely coupled designs between 54
 - sharing state 408
 - Singleton 171, 174
 - wrapping 88, 244, 254, 262, 502
 - Observer Pattern
 - about 37, 44
 - class patterns category 574
 - combining patterns 510–516
 - comparison to Publish-Subscribe 45
 - dependence in 52
 - examples of 65
 - exercise matching description of 375, 379, 596
 - in Five-minute drama 48–50
 - in Model-View-Controller 526–527, 531–533
 - loose coupling in 54
 - Observer object in 45
 - one-to-many relationships 51–52
 - process 46–47
 - Subject object in 45
 - weather station using
 - building display elements 60
 - designing 57
 - implementing 58
 - powering up 61
 - SWAG 42
 - observers
 - in class diagram 52
 - in Five-minute drama 48–50
 - in Observer Pattern 45
 - OCP (Open-Closed Principle) 355, 392
 - One Class, One Responsibility Principle. *See* Single Responsibility Principle (SRP)
 - one in ten contest in gumball machine, using State Pattern
 - about 390–391
 - annotating state diagram 396, 422
 - changing code 392–393
 - drawing state diagram 391, 420
 - implementing state classes 397, 400–405, 409
 - new design 394–396
 - reworking state classes 398–399
 - OO (Object-Oriented) design. *See* Object-Oriented (OO) design
 - Open-Closed Principle (OCP) 355, 392
 - Organizational Patterns 591
 - overusing Design Patterns 584
- ## P
- package and import statements 128
 - Participants section, in pattern catalog 571
 - part-whole hierarchy 360
 - pattern catalogs 567, 569–572
 - Pattern Death Match pages 493
 - A Pattern Language (Alexander) 588
 - patterns, using compound 493–494

- patterns, using multiple
 - about 493
 - in duck simulator
 - about rebuilding 495–497
 - adding Abstract Factory Pattern 502–505, 548
 - adding Adapter Pattern 498–499
 - adding Composite Pattern 507–509
 - adding Decorator Pattern 500–501
 - adding Iterator Pattern 507
 - adding Observer Pattern 510–516
 - class diagram 518–519
- pattern templates, uses of 573
- Pizza Store project, using Factory Pattern
 - Abstract Factory in 153, 156–157
 - behind the scenes 154–155
 - building factory 115
 - concrete subclasses in 121–122
 - drawing parallel set of classes 133, 165
 - encapsulating object creation 114–115
 - ensuring consistency in ingredients 144–148, 167
 - framework for 120
 - franchising store 118–119
 - identifying aspects in 112–113
 - implementing 142
 - making pizza store in 123–124
 - ordering pizza 128–132
 - referencing local ingredient factories 152
 - reworking pizzas 149–151
- polymorphic code, using on iterator 338, 342
- polymorphism 12
- prepareRecipe(), abstracting 284–287
- Principle of Least Knowledge 267–271. *See also* Single Responsibility Principle (SRP)
- print() method, in dessert submenu using Composite Pattern 364–367
- programming 12
- Program to an interface, not an implementation design principle 11–12, 73, 75–76, 337
- program to interface vs. program to supertype 12
- Protection Proxy
 - about 469
 - creating dynamic proxy 474–478
 - implementing matchmaking service 471–472
- protecting subjects 473
- Proxy Pattern and 466
- testing matchmaking service 479–480
- using dynamic proxy 469–470
- Prototype Pattern 612–613
- proxies 425
- Proxy class, identifying class as 480
- Proxy Pattern
 - Adapter Pattern vs. 466
 - Complexity Hiding Proxy 483
 - Copy-On-Write Proxy 483
 - Decorator Pattern vs. 466–468
 - defined 455–456
 - dynamic aspect of dynamic proxies 480
 - exercise matching description of 481, 574, 596
 - Firewall Proxy 482
 - implementation of Remote Proxy
 - about 429
 - adding to monitoring code 432
 - preparing for remote service 446–447
 - registering with RMI registry 448
 - reusing client for 449
 - reviewing process 453–454
 - role of 430–431
 - testing 450–452
 - wrapping objects and 468
 - java.lang.reflect package 440, 469, 476
- Protection Proxy and
 - about 469
 - Adapters and 466
 - creating dynamic proxy 474–478
 - implementing matchmaking service 471–472
 - protecting subjects 473
 - testing matchmaking service 479–480
 - using dynamic proxy 469–470
- Real Subject
 - as surrogate of 466
 - invoking method on 475
 - making client use Proxy instead of 466
 - passing in constructor 476
 - returning proxy for 478
- restrictions on passing types of interfaces 480
- Smart Reference Proxy 482
- Synchronization Proxy 483
- variations 466, 482–483

Virtual Proxy

- about 457
- Caching Proxy as form of 466, 482
- designing 459
- reviewing process 465
- testing 464
- writing Image Proxy 460–463

Publish-Subscribe, as Observer Pattern 45

Q

queuing requests, using Command Pattern 229

R

Real Subject

- as surrogate of Proxy Pattern 466
- invoking method on 475
- making client use proxy instead of 466
- passing in constructor 476
- returning proxy for 478

refactoring 358, 581

Related patterns section, in pattern catalog 571

relationships, between classes 22

Remote Method Invocation (RMI)

- about 432–433, 436
- code up close 442
- completing code for server side 441–444
- importing java.rmi 446
- importing packages 447, 449
- making remote service 437–441
- method call in 434–435
- registering with RMI registry 448
- things to watch out for in 444

Remote Proxy

- about 429
- adding to monitoring code 432
- preparing for remote service 446–447
- registering with RMI registry 448
- reusing client for 449
- reviewing process 453–454
- role of 430–431
- testing 450–452
- wrapping objects and 468

remove() method

- Enumeration and 252
- in collection of objects 334
- in java.util.Iterator 342

requests, encapsulating 206

resources, Design Patterns 588–589

reuse 4, 85

rule of three, applied to inventing Design Patterns 573

runtime errors, causes of 135

S

Sample code section, in pattern catalog 571

server heap 433–436

service helper (skeletons), in RMI 436–437, 440, 442–444, 453–454

shared vocabulary 26–27, 28, 585–586

Sharpen Your Pencil

- altering decorator classes 99, 108
- annotating Gumball Machine States 405, 423
- annotating state diagram 396, 422
- building ingredient factory 148, 167
- changing classes for Decorator Pattern 512, 546
- changing code to fit framework in Iterator Pattern 347, 377
- choosing descriptions of state of implementation 392, 421
- class diagram for implementation of prepareRecipe() 286, 314
- code not using factories 137, 166
- creating commands for off buttons 226, 236
- creating heat index 62
- determining classes violating Principle of Least Knowledge 270, 274
- drawing beverage order process 107
- fixing Chocolate Boiler code 183, 190
- identifying factors influencing design 84
- implementing garage door command 205, 235
- implementing state classes 402, 421
- making pizza store 124, 164
- matching patterns with categories 575–577
- method for refilling gumball machine 417, 424
- on adding behaviors 14
- on implementation of printmenu() 324, 377

- on inheritance 5, 35
 - sketching out classes 55
 - things driving change 8, 35
 - turning class into Singleton 176, 189
 - weather station SWAG 42, 75
 - writing Abstract Factory Pattern 505, 548
 - writing classes for adapters 244, 274
 - writing dynamic proxy 478, 487
 - writing Flock observer code 514, 547
 - writing methods for classes 83, 106
- Simple Factory Pattern
- about factory objects 114
 - building factory 115
 - definition of 117
 - Factory Method Pattern and 135
 - pattern honorable mention 117
 - using new operator for instantiating concrete classes 110–113
- Single Responsibility Principle (SRP) 184, 340–341, 371
- Singleton objects 171, 174
- Singleton Pattern
- about 169–172
 - advantages of 170
 - Chocolate Factory 175–176, 183
 - class diagram 177
 - code up close 173
 - dealing with multithreading 179–182, 188
 - defined 177
 - disadvantages of 184
 - double-checked locking 182
 - exercise matching description of 574
 - global variables vs. 184
 - implementing 173
 - interview with 174
 - One Class, One Responsibility Principle and 184
 - subclasses in 184
 - using 184
- skeletons (service helper), in RMI 436–437, 440, 442–444, 453–454
- smart command objects 228
- Smart Reference Proxy 482
- software development, change as a constant in 8
- sorting methods, in Template Method Pattern 302–307
- sort() method 306–311
- spliterator method 343
- SRP (Single Responsibility Principle) 184, 340–341, 371
- Starbuzz Coffee Barista training manual project
- about 278–283
 - abstracting prepareRecipe() 284–287
 - using Template Method Pattern
 - about 288–290
 - code up close 292–293
 - defined 291
 - The Hollywood Principle and 298–300
 - hooks in 293–295
 - testing code 296
- Starbuzz Coffee project, using Decorator Pattern
- about 80–81
 - adding sizes to code 99
 - constructing drink orders 89–90
 - drawing beverage order process 94, 107
 - testing order code 98–99
 - writing code 95–97
- state machines 384–385
- State Pattern
- defined 406
 - exercise matching description of 418, 424, 574, 596
 - gumball machine controller implementation
 - cleaning up code 413
 - demonstration of 411–412
 - diagram to code 384–385
 - finishing 410
 - refilling gumball machine 416–417
 - SoldState and WinnerState in 412
 - testing code 388–389
 - writing code 386–387
 - increasing number of classes in design 408
 - modeling state 384–385
 - one in ten contest in gumball machine
 - about 390–391
 - annotating state diagram 396, 422
 - changing code 392–393
 - drawing state diagram 391, 420
 - implementing state classes 397, 400–405, 409
 - new design 394–396
 - reworking state classes 398–399
 - sharing state objects 408
 - state transitions in state classes 408
 - Strategy Pattern vs. 381, 407, 414–415
 - state transitions, in state classes 408

state, using to implement undo commands 221

static classes, using instead of Singletons 184

static method vs. create method 115

Strategy Pattern

- algorithms and 24
- defined 24
- exercise matching description of 300, 314, 375, 379, 418, 424, 574, 596
- in Model-View-Controller 526–527, 536–537, 539
- State Pattern vs. 381, 407, 414–415
- Template Method Pattern and 307

strive for loosely coupled designs between objects that interact design principle 54. *See also* Loose Coupling Principle

structural patterns category, Design Patterns 576, 578–579

Structure section, in pattern catalog 571

stubs (client helper), in RMI 436–437, 440, 442–444, 453–454

subclasses

- concrete commands and 207
- concrete states and 406
- explosion of classes 81
- Factory Method and, letting subclasses decide which class to instantiate 134
- in Singletons 184
- Pizza Store concrete 121–122
- Template Method 288
- troubleshooting 4

Subject

- in class diagram 52
- in Five-minute drama 48–50
- in Observer Pattern 45–47

subsystems, Facades and 262

superclasses 4, 12

supertype (programming to interface), vs. programming to interface 12

Swing library 65, 308, 543

synchronization, as overhead 180

Synchronization Proxy 483

T

Template Method Pattern

- about 288–290
- abstract class in 292, 293, 297
- applets in 309
- class diagram 291
- code up close 292–293
- defined 291
- exercise matching description of 300, 314, 418, 424, 574, 596
- The Hollywood Principle and 298–300
- hooks in 293–295, 297
- in real world 301
- sorting with 302–307
- Strategy Pattern and 307
- Swing and 308
- testing code 296

thinking in patterns 580–581

tightly coupled 54

The Timeless Way of Building (Alexander) 588

transparency, in Composite Pattern 371

tree structure, Composite Pattern 360–362, 368

Two Way Adapters, creating 244

type safe parameters 135

U

undo commands

- creating 217–219, 228
- implementing for macro command 228
- support of 217
- testing 220, 223–224
- using state to implement 221

User Interface Design Patterns 591

V

variables

- declaring behavior 15
- holding reference to concrete class 143
- instance 82–83, 97–98

Vector 352

Virtual Proxy

about 457

Caching Proxy as form of 466, 482

designing 459

reviewing process 465

testing 464

writing Image Proxy 460–463

Visitor Pattern 614–615

Vlissides, John 587–588

volatile keyword 182

W

weather station

building display elements 60

designing 57

implementing 58

powering up 61

Who Does What exercises

matching objects and methods to Command Pattern
202, 235

matching patterns with its intent 256, 275

matching pattern with description 300, 314, 375, 379,
418, 424, 481, 488, 574, 596

whole-part relationships, collection of objects using 372

Wickedlysmart website xxxiii

wrapping objects 88, 244, 254, 262, 468, 502

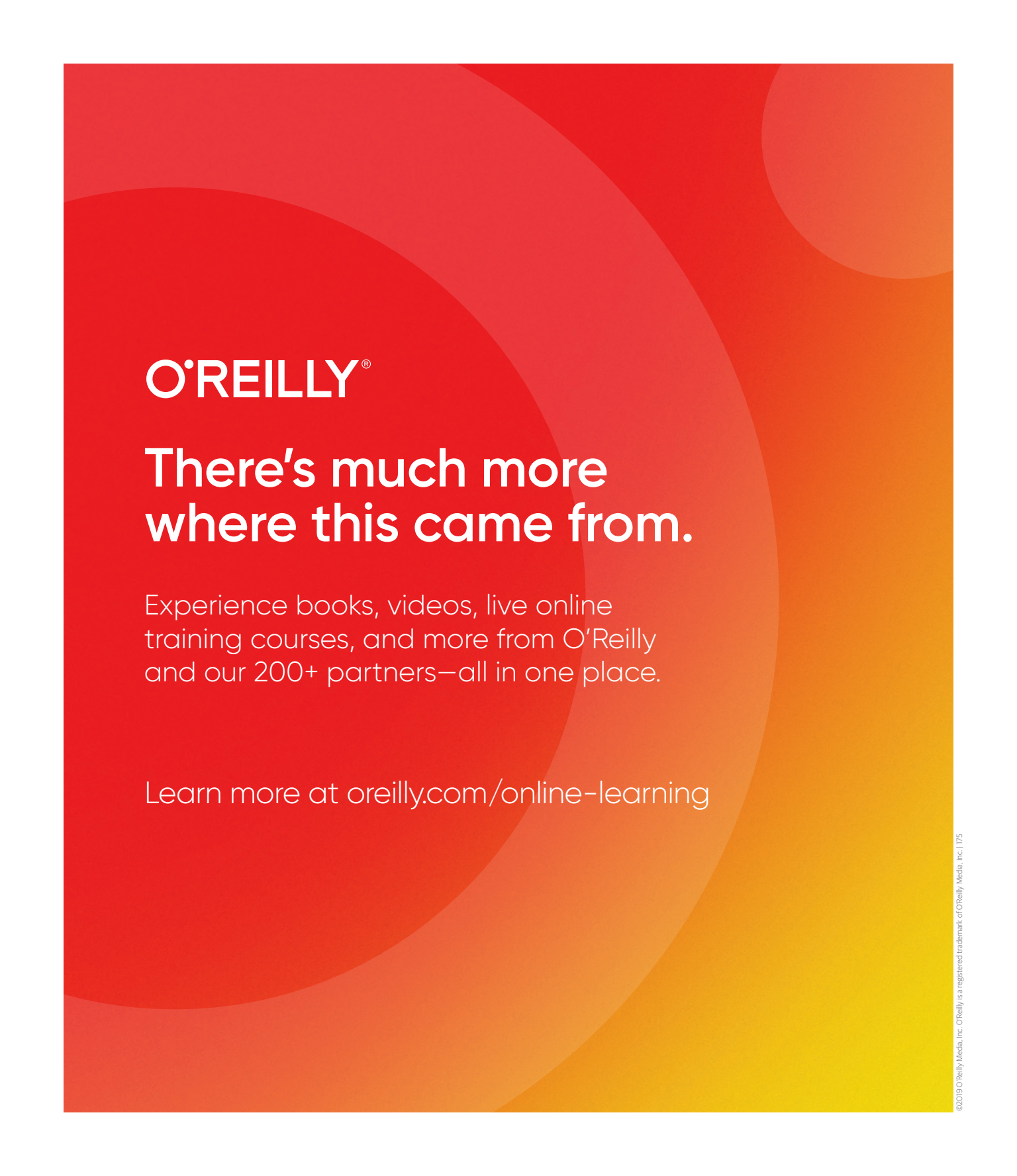
Y

your mind on patterns design pattern 583



Don't know about the website?
We've got updates, video,
projects, author blogs, and more!

**Bring your brain over to
wickedlysmart.com**

The background of the entire page is a vibrant red-to-orange gradient. Overlaid on this are several large, semi-transparent circles in varying shades of red and orange, creating a layered, organic effect.

O'REILLY®

There's much more where this came from.

Experience books, videos, live online training courses, and more from O'Reilly and our 200+ partners—all in one place.

Learn more at oreilly.com/online-learning